

NammaPG – Authentication Process Notes

Overview

These notes document the complete authentication process designed for the NammaPG backend application using Spring Boot, Spring Security, and JWT. The objective is to build a secure, scalable, stateless authentication system suitable for real-world production use.

Technology Stack

- 1 Spring Boot 3.2.5 (Stable release)
- 2 Java 17
- 3 Spring Security 6
- 4 JWT (JSON Web Tokens)
- 5 Spring Data JPA
- 6 MySQL Database
- 7 Maven build tool

Authentication Architecture

The authentication system is stateless and token-based. Once a user logs in successfully, the server generates a JWT token. This token is sent with every subsequent request in the Authorization header. No server-side sessions are maintained.

Roles Used

- 1 PG_USER – Regular PG residents
- 2 PG_OWNER – PG owners / managers
- 3 ADMIN – Platform administrators

Phase 0: Project Setup

The project was initialized with Spring Boot 3.2.5 and Java 17. Major issues faced during setup included dependency resolution errors due to invalid Spring Boot versions (3.9, 3.5.x SNAPSHOT). These were resolved by switching to the stable 3.2.5 release and clearing Maven and IntelliJ caches.

Phase 1: Project Structure & Repositories

The codebase is organized into clear packages such as entity, repository, security, controller, and payload. Repositories are designed to query users by email and roles by name, which aligns with Spring Security best practices.

Phase 2: JWT Security Components (Planned)

- 1 JwtUtils – Responsible for generating, validating, and parsing JWT tokens.
- 2 AuthTokenFilter – Intercepts every HTTP request, extracts JWT tokens, and authenticates users.

- 3 AuthEntryPointJwt – Handles unauthorized access errors (HTTP 401).

Phase 3: Security Configuration (Completed)

The SecurityConfig class configures Spring Security for stateless JWT authentication. CSRF is disabled, session management is set to STATELESS, public and protected endpoints are defined, and BCryptPasswordEncoder is registered for password hashing.

Phase 4: Authentication APIs (Planned)

- 1 POST /api/auth/signin – Authenticates user credentials and returns JWT.
- 2 POST /api/auth/signup – Registers a new user and assigns roles.
- 3 DTOs are used to separate request and response payloads.

Phase 5: Role Data Seeding (Planned)

On application startup, a RoleInitializer will ensure that all required roles (PG_USER, PG_OWNER, ADMIN) exist in the database. This prevents signup failures on fresh deployments.

Conclusion

At this stage, the authentication foundation is fully designed and partially implemented. The project is now ready to proceed with repository implementation, JWT logic, and authentication controllers. These notes serve as a reference for future development and revision.